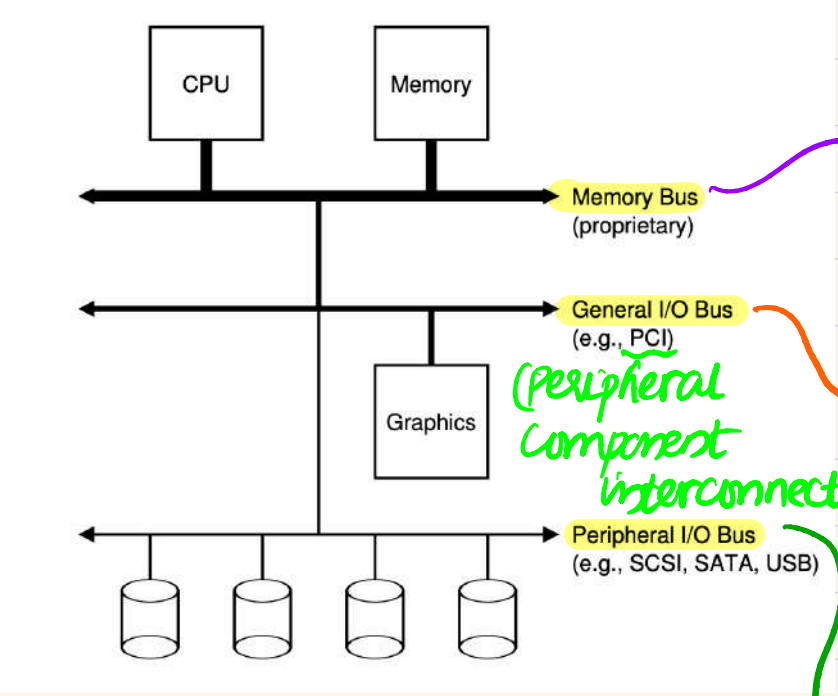
I/O devices

Question: How does the OS interacts with I/O devices?



A typical system architecture

→ connects the CPU and main memory directly.
- extremely high-speed and low-latency
- optimized for memory access.

→ connects CPU/memory subsystems to high speed I/O devices.
- provides a more general interface for devices that need high band width eg:- graphics card, disk controllers, network interface cards, etc.

(Peripheral Component interconnect)

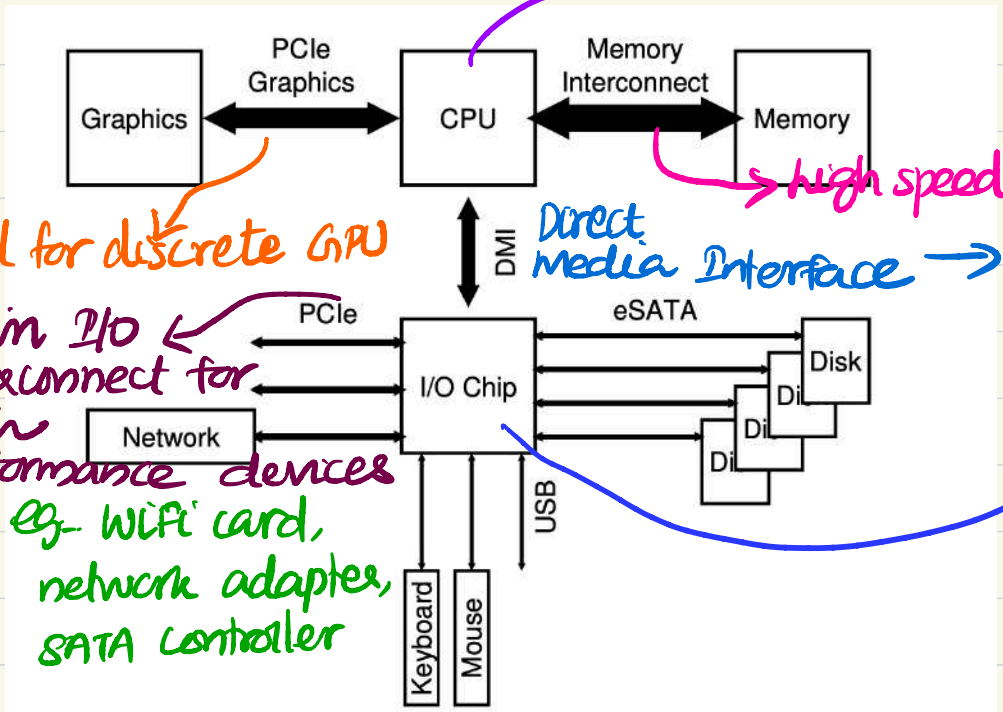
serial AT attachment

a slower, more flexible bus for lower-speed, such as SCSI, SATA or USB. It connects slow devices to the system.
eg:- disks, mouse, keyboard, etc.

(Small computer System Interface)

Why this hierarchy exists?

* Speed separation: memory accesses (nano seconds) Vs disk access (milli seconds) - very different speeds; the faster a bus is, the shorter it must be.
* Flexibility: different device types can attach via their appropriate buses. eg:- components that demand high performance needs to be nearer CPU.
* Scalability: you can add new bus technologies (USI, PCIe, Thunderbolt) without redesigning the whole system.



→ contains more than just cores.
- Integrated memory controller
- Integrated PCI Express controller.

→ high speed connection directly to RAM

used for discrete GPU

Direct Media Interface → carries all traffic b/w CPU and I/O devices.

main I/O interconnect for high performance devices eg- WiFi card, network adapter, SATA controller

→ central hub for all slower and peripheral I/O.

SATA (serial ATA)
→ interface for internal storage drives (SSDs / HDDs)

eSATA: External version for removable storage

modern system architecture: Intel Z270 chipset

Why this modern design?

* Parallelism: multiple independent links (PCIe lanes) allow simultaneous transfers.
* Performance and efficiency: DMA reduce CPU workload
* Scalability: add devices without affecting others

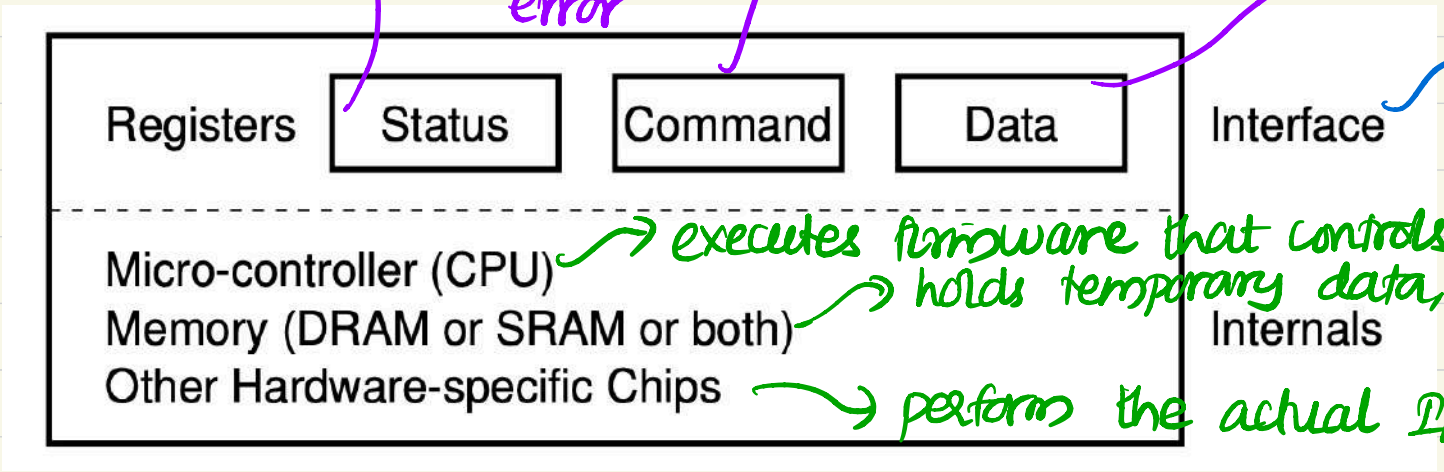
A canonical device

OS must work with many different devices - disks, keyboards, network cards, GPUs, etc. So, its impossible for the OS to know each one's hardware details.

The canonical device model → gives the OS a standard mental model to interact with any device.

It defines
- What's inside a device?
- How the CPU talks to it?
- What role each part plays?

This lets the OS design generic mechanisms that can support all devices, no matter their specific hardware.



A canonical device

(eg:- sending signals, reading bits, etc.)

* firmware → software within a hardware device to implement its functionality.

Canonical Protocol:

↳ the standard sequence of steps that the CPU (or OS) and an I/O device follow to perform an operation.

eg:- reading from a disk, sending a network packet etc..

Why canonical? → this sequence (or protocol) is common to all devices, regardless of their specific details.

The four steps in the Protocol:

```
While (STATUS == BUSY)
    ; // wait until device is not busy
Write data to DATA register
Write command to COMMAND register
    (starts the device and executes the command)
While (STATUS == BUSY)
    ; // wait until device is done with your request
```

1. wait for device to be ready
or CPU? OS repeatedly reads the status register to check if the device is ready to accept a new command.
   → this constant checking is called polling.

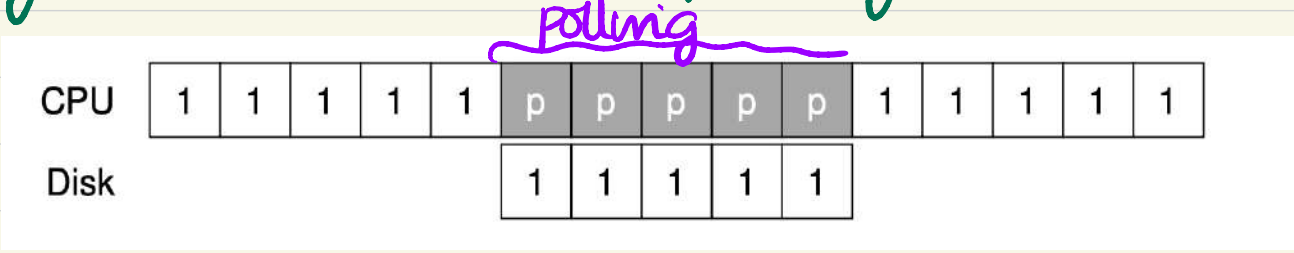
2. OS send data to the device
   → when the main CPU is involved in the data movement, we refer to it as a programmed I/O (PIO)

3. OS issue the command
   → let the device know all set up is complete and it can start processing the data.

4. wait for completion (polling again)
   → read the status register until it reports completion.

Limitation:

Polling is inefficient → the CPU wastes time repeatedly checking the device instead of doing other work.

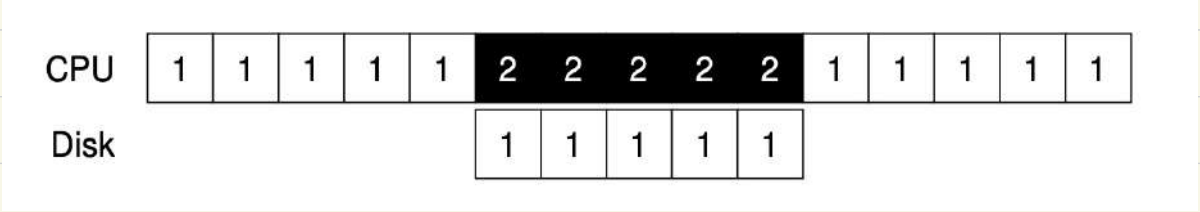


So what to do?

Lowering CPU overhead with interrupts

Idea: instead of polling the device repeatedly,
OS raise a request → put calling process to sleep → context switch to other task.

When device is finally finished
raise a hardware interrupt → OS executes interrupt handler → wake the process waiting for the I/O.



→ interrupts allow for overlap of computation and I/O

Are interrupts always a best solution? Not necessarily

for eg:- (1) when device is faster, context switch incur more cost.

(2) if there is a huge stream of incoming packets, it causes a livelock → find itself only processing interrupts and not doing actual work.

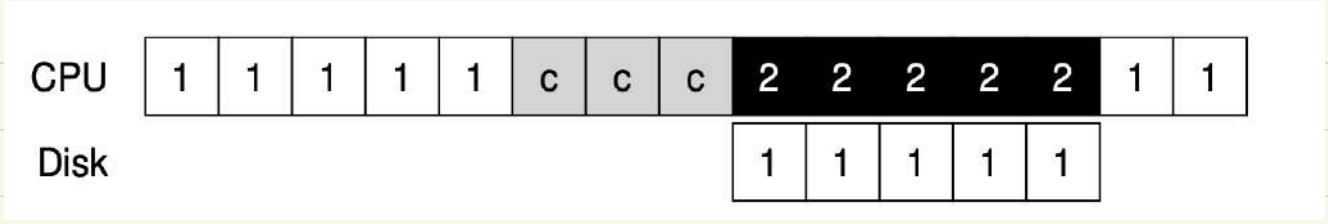
That's why modern systems combine both:

- use polling when events are frequent and predictable.
- use interrupts for slower and unpredictable devices.

Another interrupt-based optimization → coalescing

- multiple interrupts can be coalesced into a single interrupt delivery, thus lowering the overhead of interrupt processing.

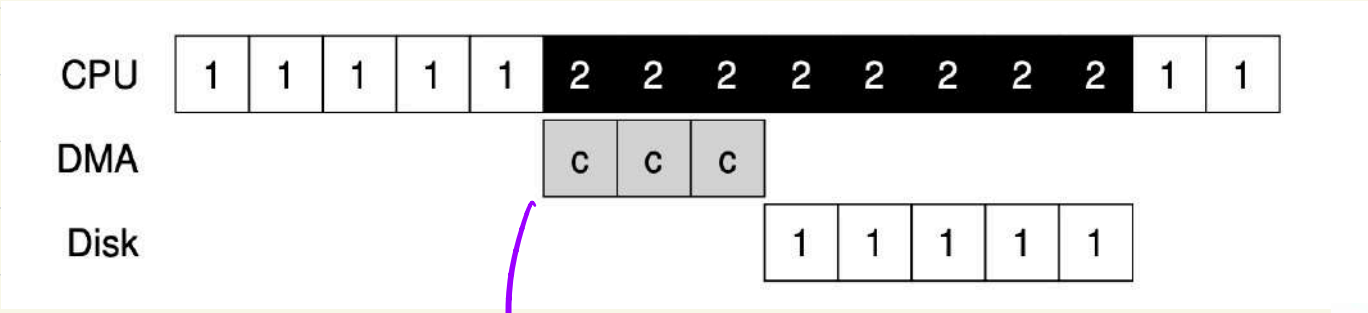
Another problem in canonical protocol → programmed I/O.



→ CPU is overburdened with copying data
→ wastage of CPU time

What is the solution? Direct Memory access (DMA)

a specific device that performs transfer b/w devices and main memory without much CPU intervention.



DMA raises an interrupt
→ OS knows transfer is complete

→ OS tells DMA engine
- memory location of data
- how much data to copy
- which device to send it to

Canonical protocol → what happens logically when the OS interacts with the device.

Question: what is the hardware mechanism that actually makes this communication happen?

- The OS needs to
  - write commands to device registers
  - Read device status
  - transfer data to/from data buffers.

So, we need a hardware-level communication mechanism b/w CPU and device that supports
  - reading/writing registers
  - exchanging data safely and efficiently.

Two main approaches

1. special I/O instructions: some architectures (like X86) have dedicated assembly instructions for device communications such as

```
in <destination register>, <port number>
```

into CPU register AL

eg:- `in al, 0x60`

Read one byte from I/O port (eg:- keyboard)

These use a seperate I/O address space (called port-mapped I/O). Each device register is given a unique I/O port number and only

privileged instructions can access them.

Pros:
- keeps device instructions separate from memory addresses
- clear distinction b/w memory and I/O operations.

Cons
- Requires special instruction → not portable
- Adds complexity to CPU design.

2. Memory mapped I/O (MMIO): A more modern and common approach where the device's control registers are mapped into the system's physical address space so that the OS can read/write them using normal load/store instructions (like accessing memory).

eg:-
```
*(volatile int *)0xC0000000 = 1;  // write to device command register
```

When the CPU writes to that memory address, it actually sends the signal over the bus to the device, not to RAM.

Pros:
- uses standard instructions (no special CPU support needed)
- simplifies system design - devices behaves like special memory
- easier for modern CPUs and compilers to handle.

Cons
- needs careful mapping to avoid confusion with real memory
- accesses must be marked as volatile to prevent compiler optimization

Question: who actually writes the code inside the OS that does all this device-specific interaction?

In fact, different hardware devices (disks, keyboards, printers, etc.) all
- have their own registers
- speak different protocols, and
- behave differently at the hardware level.

Yet the OS kernel wants to expose a uniform interface to user programs. eg:- system calls like

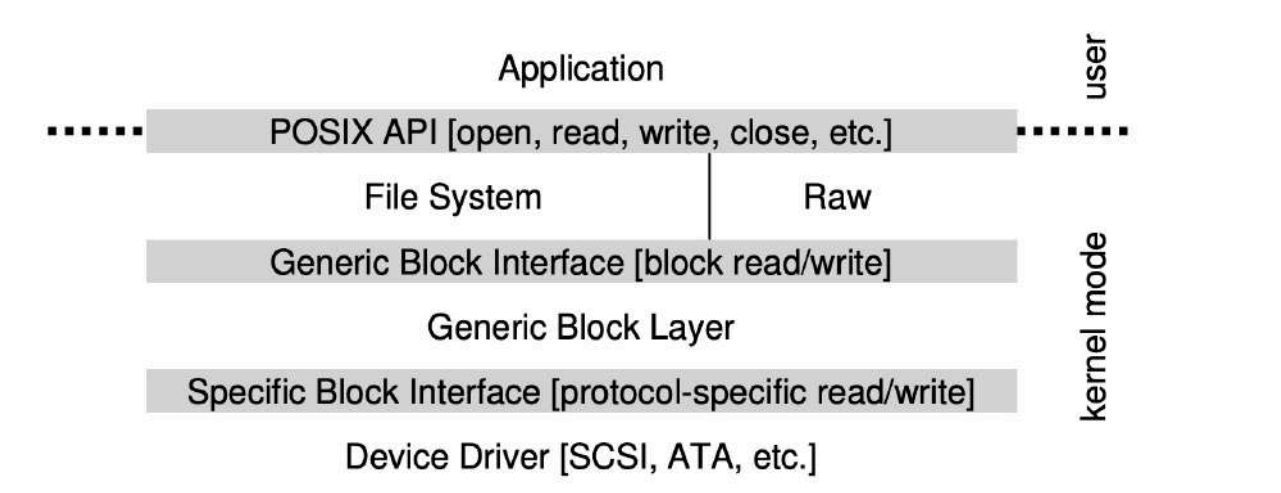
```
read(fd, buf, n);
write(fd, buf, n);
ioctl(fd, ...);
```

How can it happen if devices are all so different?

↳ due to Device driver!

a piece of kernel code that acts as a translator b/w
- the generic OS interface (eg:- read, write, open, close, etc.) and
- the special hardware operations (eg:- writing to registers, handling interrupts)

Device driver → piece of software in the OS who knows in detail how a device works.



a depiction of Linux file system software stack.

eg - writing a file to disk

1. user calls write(fd, buf, 4096)
2. Generic OS interprets the file descriptor → maps to a disk device
3. Driver: sends commands to the disk controller via MMIO or DMA
4. Hardware: performs the operation and signals completion (interrupt)

Why it's needed?

* Portability: The OS kernel doesn't need to know the device specific details. You can add new devices without re-writing the kernel
* Modularity: Drivers are separate modules that can be loaded/unloaded dynamically.

To summarize, a device driver is the glue b/w the OS high-level I/O interface and the hardware's low-level control interface.
- It hides hardware details, making the OS portable and modular.